

Seu Cantinho - Trabalho de Design de Software

Nathália Nogueira

02/2025

O objetivo deste relatório é descrever o desenvolvimento do trabalho da disciplina Design de Software, desde a definição de arquitetura e diagramas UML até a codificação e documentação.

O código-fonte, diagramas e documentações estão disponíveis em:

<https://github.com/nathalia-nogueira/seu-cantinho.git>

1. Solução proposta

a. Estilo arquitetural

O estilo definido foi o **cliente-servidor com arquitetura em camadas**. O cliente, como apenas faz requisições, será implementado em um bloco monolítico que contém o *frontend*, enquanto o servidor será dividido nas camadas **Modelo**, **Serviço**, **Controlador** e **Repositório**. Essa arquitetura se destaca para o cenário de *Seu Cantinho* por sua:

- **Simplicidade:** facilidade de desenvolvimento, testes e, principalmente, implantação.
- **Escalabilidade:** mesmo que outros estilos arquiteturais (como microsserviços) ofereçam maior escalabilidade, a escalabilidade horizontal do estilo é mais do que o suficiente para o cenário de três estados e consideravelmente mais simples de implementar.
- **Facilidade de manutenção:** garantida pela modularização do código em camadas.

Com relação a tradeoffs identificados, há um grande ganho em simplicidade de desenvolvimento e implantação, além de consistência nas transações, enquanto escalabilidade e isolamento de falhas são colocados em segundo plano.

b. Diagramas UML

No desenvolvimento de *Seu Cantinho*, foram criados os diagramas UML de **classes** e de **componentes**. Ambos podem ser encontrados abaixo; contudo, para uma visualização mais clara e detalhada, recomenda-se acessá-los diretamente no repositório do *Github*.

a. Diagrama de Classes

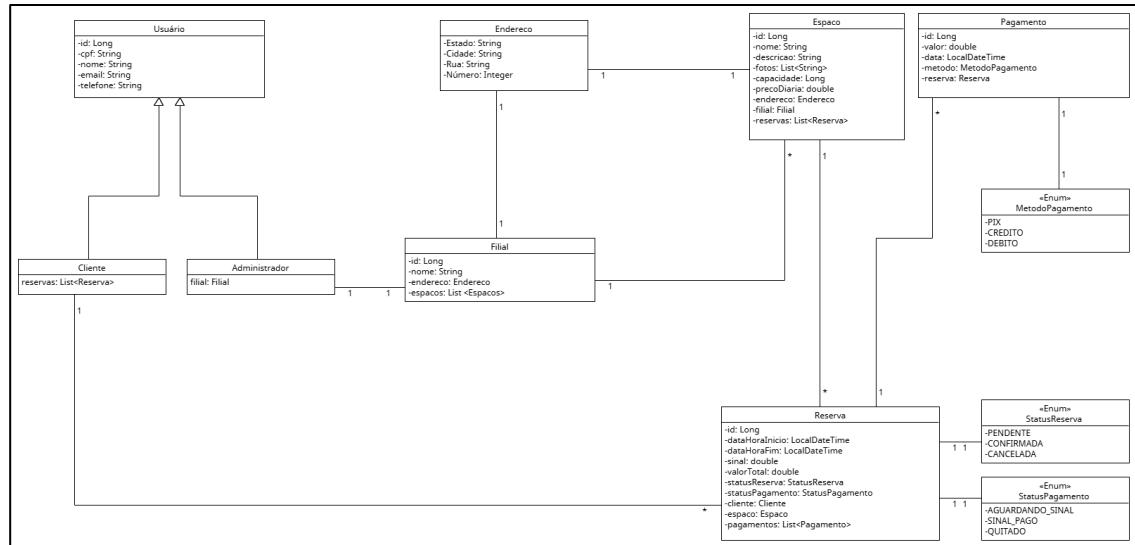


Figura 1 – Diagrama de Classes para Seu Cantinho

É importante destacar que, na figura 1, apenas a camada **Modelo** foi representada. As demais camadas não foram incluídas para evitar redundância, visto que serão compostas por classes correspondentes às principais entidades - usuário, espaço, reserva e pagamento. A responsabilidade de cada uma das camadas e, consequentemente, de suas respectivas classes, está descrita na seção 2.b.

Além disso, observa-se que as classes estruturadas dessa forma caracterizam um **modelo anêmico** - classes que contêm apenas atributos, sem métodos associados. Ainda assim, é a solução adequada para o problema do *Seu Cantinho*, pois permite a utilização de transações atômicas (evitando o problema de double-booking) e conta com simplicidade e clareza úteis nesse contexto.

b. Diagrama de Componentes

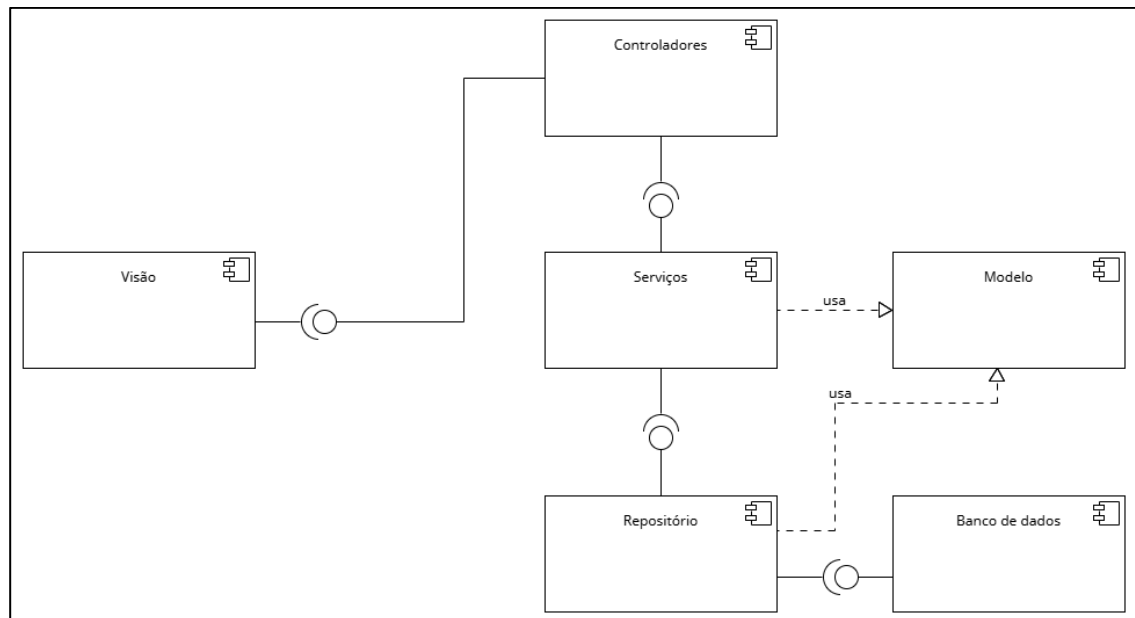


Figura 2 – Diagrama de Componentes para Seu Cantinho

Como representado na figura 2, a arquitetura é dividida nos seguintes componentes:

1. **Visão**: interface com o usuário que consome a API REST
2. **Controladores**: recebe requisições HTTP e as direciona para a camada de Serviços
3. **Serviços**: contém a lógica de negócio central
4. **Repositório**: define as interfaces de acesso aos dados – traduz as chamadas dos Serviços em consultas ao Banco de Dados
5. **Banco de dados**: garante persistência e atomicidade das transações

2. Aderência da solução aos requisitos

A modelagem apresentada foi definida para solucionar os problemas operacionais relatados no uso do sistema anterior. A própria estrutura de classes do Modelo já endereça os requisitos funcionais básicos:

- **Múltiplas filiais**: a expansão para três estados é tratada pela classe Filial, que se relaciona com Espaço e Administrador. Isso permite que o sistema

gerencie espaços e administradores por localidade, mantendo a consistência entre filiais.

- **Cadastro dos espaços:** feito pela entidade Espaço
- **Gerenciamento de reservas:** fornecido pela classe Reserva
- **Controle de pagamentos:** modelado por Pagamento e StatusPagamento

Contudo, a questão mais crítica era o **double-booking**, principal queixa de Dona Maria. A prevenção de reservas conflitantes é uma responsabilidade da camada de Serviço, que segue o fluxo:

1. O usuário (via Visão) solicita uma nova reserva, recebida pelo Controlador
2. O Controlador chama a camada de Serviço, que é responsável pela lógica de negócio. Antes de salvar a nova reserva, o Serviço usa o Repositório para consultar o Banco de Dados e verificar se existe reserva conflitante.
3. Se a consulta retornar um conflito, o Serviço cancela a operação e informa o usuário. Se não houver conflito, o Serviço cria a reserva.

Para garantir a integridade em casos de requisições simultâneas, todo esse processo é executado em uma **transação atômica** do banco de dados. Isso garante que, se duas requisições conflitantes chegarem ao mesmo tempo, o mecanismo de lock do banco fará com que a primeira finalize a operação antes que a segunda possa verificar a disponibilidade. A segunda, então, “enxergará” a reserva criada pela primeira e será devidamente rejeitada.

3. Tecnologias escolhidas

Para a implementação, as tecnologias foram selecionadas visando o equilíbrio entre a robustez necessária para Seu Cantinho e a simplicidade de configuração para o protótipo. As escolhas se dão por:

- **Backend:** a linguagem escolhida foi o **Java**, aproveitando a familiaridade da desenvolvedora e a naturalidade na tradução dos Diagramas de Classes UML para o código orientado a objetos. Como framework web, optou-se pelo **Spark (SparkJava)** - diferente de frameworks mais robustos e complexos, o Spark oferece uma estrutura minimalista e leve, facilitando a criação rápida das rotas e controladores sem a necessidade de configurações extensas.
- **Banco de Dados:** para a persistência, foi escolhido o **PostgreSQL** como banco relacional. A comunicação com o banco é mediada pelo **Hibernate**. O uso deste framework ORM foi definido para simplificar o desenvolvimento,

permitindo que as entidades do Modelo fossem mapeadas diretamente para tabelas, abstraindo a complexidade de consultas SQL manuais.

- **Frontend:** a interface com o usuário foi construída utilizando apenas **HTML, CSS e JavaScript** nativos, sem o uso de frameworks de frontend. Essa decisão removeu a complexidade de gerenciamento de estado, mantendo a arquitetura monolítica simples e focada na integração direta com o backend.

4. Funcionalidades pendentes

A especificação do projeto inclui duas funcionalidades que, por questões de prazo, não foram refletidas na interface do usuário (frontend). No entanto, ambas já se encontram devidamente estruturadas no backend, o que garante uma base sólida e agilidade para sua implementação visual em etapas futuras.

- **Fluxo de pagamento:** embora a interface exiba a opção "Pagar", a funcionalidade não completa o ciclo de processamento na versão atual. A base lógica está totalmente construída: a classe de domínio Pagamento já está modelada e já existem métodos que tratam o caso de uso. A pendência se dá estritamente pela falta de integração entre frontend e backend.
- **Galeria de fotos dos espaços:** a visualização e o cadastro de imagens dos espaços não foram finalizados na camada de apresentação. No entanto, o suporte a essa funcionalidade já existe no nível de dados: a entidade Espaço possui o mapeamento para uma lista de fotos, permitindo que o banco de dados armazene e recupere esses caminhos assim que sejam implementados na interface.

5. Limitações do protótipo e próximos passos

O desenvolvimento do projeto focou na entrega de um MVP (Produto Viável Mínimo) que validasse a arquitetura em camadas e a lógica central de reservas. Para a evolução do sistema, foram identificados os seguintes pontos de melhoria e débitos técnicos a serem solucionados:

- **Padronização dos Controladores (Reservas e Endereços):** a implementação das requisições GET e POST para as entidades Reserva e Endereço foi realizada de forma manual e imperativa, divergindo dos padrões de abstração aplicados nas outras entidades. Assim, o próximo passo nesse contexto é refatorar esses controladores para que utilizem a

mesma estrutura elegante do restante do sistema, facilitando a manutenção futura.

- **Tratamento de Erros e Exceções:** o sistema atual possui um tratamento de erros básico - por exemplo, ao tentar cadastrar uma reserva conflitante, o único retorno é o código HTTP 500, sem mais indicações. É necessária a implementação de um manipulador de exceções para capturar falhas inesperadas e retornar mensagens amigáveis e padronizadas (com códigos HTTP corretos) tanto para a API quanto para o usuário final.
- **Validação de Dados:** atualmente, a validação de campos é simplificada. A evolução do software requer a inclusão de regras de validação robustas tanto no frontend quanto no backend, garantindo a integridade dos dados antes que cheguem ao banco.
- **Interface de Edição (Update):** embora o backend suporte as operações de atualização (o "U" do CRUD), a interface gráfica atual não expõe telas para edição de cadastros. A implementação dessas telas é necessária para dar total autonomia aos administradores.
- **Integração real de pagamentos:** o sistema atual apenas simula a transação. O próximo passo envolve a integração com um *gateway* de pagamento externo.
- **Segurança e Autenticação:** O protótipo utiliza um controle de acesso simplificado. A versão final requer a implementação de padrões como OAuth2 ou JWT, além de criptografia de dados sensíveis.
- **Observabilidade:** para garantir a confiabilidade em produção, deve-se implementar *logs* estruturados e ferramentas de monitoramento para rastrear a performance e diagnosticar falhas rapidamente.

6. Conclusão

O desenvolvimento do **Seu Cantinho** cumpriu seu objetivo principal de conectar a teoria vista em sala à prática, demonstrando como uma arquitetura bem definida guia a implementação de um sistema robusto. A solução entregue valida os diagramas UML propostos e soluciona o problema crítico de double-booking através de um gerenciamento eficiente de transações e camadas.

Embora o sistema apresente limitações técnicas naturais de um protótipo, ele estabelece uma base sólida, funcional e extensível, evidenciando a viabilidade da arquitetura Cliente-Servidor adotada.